

Голосовой AI-Ассистент с web-поиском (Групповой проект)

В этом проекте вам предстоит создать полноценного голосового ИИ-ассистента, способного не только общаться с пользователем голосом, но и взаимодействовать с веб-сайтами для поиска информации с помощью **MCP Playwright**.

Это комплексная задача, которая объединяет в себе работу с аудио-моделями (Speech-to-Text, Text-to-Speech), построение агентных систем на базе LLM (**LangChain**), использование **Model Context Protocol (MCP)** и классическую Fullstack-разработку.

Основная информация

- **Размер команды:** 3-4 человека
- **Длительность:** [Укажите дедлайн]
- **Стек технологий (рекомендованный):** Python (FastAPI / LangChain), React / Vue / Vanilla JS для Frontend, Playwright, Whisper (или аналоги), ElevenLabs.

Цель проекта: Voice-Ассистент по Планированию Досуга

Задача: Разработать веб-приложение искусственного интеллекта, с которым можно общаться исключительно голосом (но с дублированием текстом на UI).

Главный Use-Case агента: Помощник для планирования дня, вечера или выходных. Агент должен уметь искать билеты, проверять расписание мероприятий, кино, выставок и предлагать пользователю варианты.

Как это должно работать (User Flow):

1. Пользователь нажимает кнопку и говорит в микрофон: *"Привет! Куда можно сходить сегодня вечером в Алматы? Желательно на концерт или стендап."*
2. Система принимает аудио и переводит голос в текст (**ASR**).
3. Текст передается **LLM-агенту** (на базе LangChain).
4. Агент автономно понимает, что ему нужно поискать свежую веб-афишу. Он использует инструмент **MCP Playwright**, чтобы зайти на сайт, например, sxodim.com/almaty, kino.kz, ticketon.kz (или любые другие сайты, парсинг которых вы настроите), читает страницу и собирает актуальные мероприятия на вечер.
5. Агент генерирует финальный текстовый ответ.
6. Текстовый ответ переводится в речь (**TTS**) и автоматически проигрывается пользователю: *"Сегодня вечером есть классный стендап на Абая в 19:00 и джазовый концерт в 20:00. Куда бы ты хотел?"*

Технические требования и Рекомендуемые Модели

1. Распознавание речи (ASR / STT)

- **Реализация:** Необходимо перевести аудио от пользователя в текст.
- **Облачные API:** OpenAI Whisper API, Google Speech-to-Text, AssemblyAI.
- **Локальные модели:**
 - [nvidia/parakeet-tdt-0.6b-v3](https://nvidia.github.io/parakeet-tdt-0.6b-v3) — отличная производительность и скорость.
 - [openai/whisper-large-v3-turbo](https://openai.com/whisper-large-v3-turbo) — оптимизированная версия самой мощной модели Whisper.
 - [Systran/faster-whisper-small](https://systran.github.io/faster-whisper-small) — CTranslate2 имплементация Whisper, работает в разы быстрее оригинальной библиотеки при минимальном потреблении памяти.

[!TIP]



Подсказки по гиперпараметрам ASR (на базе Whisper):

- **beam_size (размер луча):** Стандартно 5. Уменьшение ускорит инференс (например, до 1, так называемый *greedy search*), но может слегка снизить качество.
- **temperature:** Оставьте `0.0` для максимально точного распознавания. Если модель начинает галлюцинировать или "зацикливаться", можно попробовать массив значений (например, `[0.0, 0.2, 0.4]`), чтобы модель искала более вероятные пути.
- **condition_on_previous_text:** Если модель "залипает" на повторении последней сказанной фразы (что бывает на длинных аудио), попробуйте выставить в `False`. Вы потеряете небольшой процент контекста, но повысите стабильность распознавания.

2. Синтез речи (TTS)

- **Реализация:** Текстовый ответ агента должен звучать естественно.
- **Облачные API: ElevenLabs API** (предоставляет лучший голос и поддержку стриминга с минимальной задержкой), OpenAI TTS.
- **Локальные / Open-Source модели:**
 - fishaudio/s2-pro (Fish Speech) — превосходное качество речи с поддержкой клонирования голоса.
 - coqui-ai/XTTS-v2 — отлично поддерживает русский и английский, также имеет voice cloning и потоковую генерацию.
 - suno/bark или современные аналоги вроде **OuteTTS** (если позволяют ресурсы видеокарты).

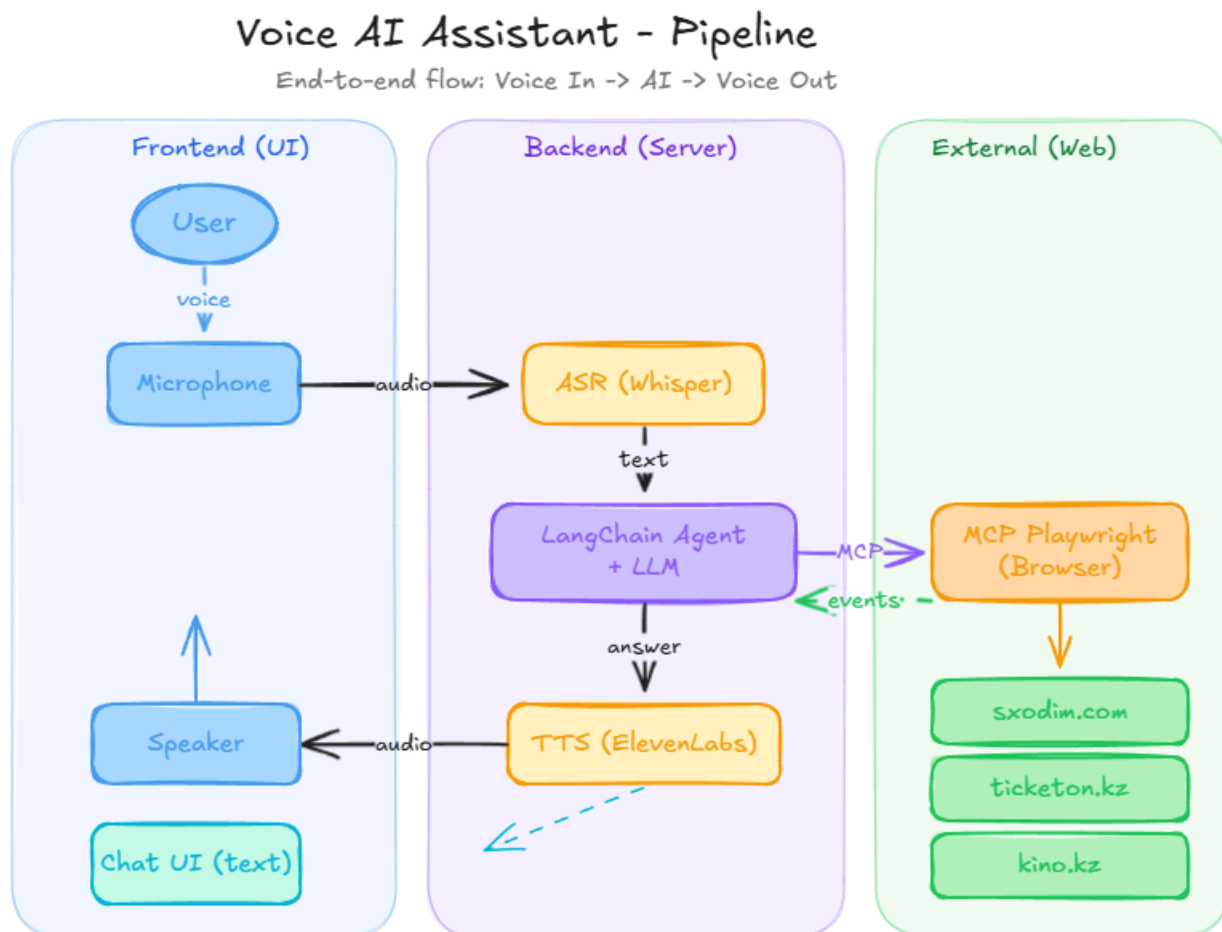
3. Мозг ассистента: LangChain Агент + MCP Playwright

- **Основа:** Вы должны собрать агента, используя **LangChain**.
- **Инструменты (Tools):** Ключевое требование — агент должен использовать **MCP сервер** для Playwright.
- **Поведение:** Агент должен быть достаточно "умным", чтобы понимать, *на какие конкретно веб-сайты (Афиши)* ему нужно пойти на основе запроса

пользователя. Вам предстоит составить список целевых сайтов-источников для агента и научить его искать по ним актуальные ивенты.

4. Fullstack обертка (Frontend + Backend)

- **Backend:** Сервер, который оркестрирует все процессы. Он должен принимать аудио, вызывать ASR, передавать текст агенту с MCP, отдавать сгенерированный текст в TTS и возвращать аудиофайл или стрим на клиент.
- **Frontend:** Современный, минималистичный пользовательский интерфейс. Кнопка для записи голоса, красивая анимация во время раздумий агента и воспроизведения аудио.



✓ Критерии оценки (Definition of Done)

1. **End-to-end флоу работает:** Можно спросить что-то голосом и получить ответ голосом.
 2. **MCP Playwright интегрирован:** Агент успешно выполняет запросы, требующие серфинга в интернете (ищет мероприятия на целевых сайтах вроде sxodim.com).
 3. **Качество аудио:** Нормальная работа с ASR и качественный голос от TTS.
 4. **Организация кода:** Код структурирован, Backend и Frontend разделены, API логично.
 5. **Командная работа:** Использование Git (ветвления, PR-ы), регулярные коммиты от всех участников.
-

🌟 Бонусное задание (Advanced Level / Реал-тайм)

Для получения дополнительных баллов и создания системы, максимально отзывчивой на действия пользователя:

1. **Real-time ASR (через WebSocket):** Поднимите легковесную локальную модель (например, `faster-whisper-small`) и реализуйте передачу аудио с фронта на бэкенд через **WebSocket**. Это позволит транскрибировать речь и показывать текст на экране в режиме реального времени, пока пользователь еще продолжает говорить.
2. **Streaming TTS:** Настройте стриминг аудио обратно на фронтенд. Используйте потоковую передачу данных (**ElevenLabs API Streaming** или `stream` через локальную модель, такую как XTTS-v2 / Fish Speech), чтобы ответ агента начинал проигрываться пользователю по мере генерации текста агентом, не дожидаясь формирования полного ответа.